

8. Production

Putting a Python program to use requires moving it from a development environment to a production environment. Supporting disparate configurations like this can be a challenge. Making programs that are dependable in multiple situations is just as important as making programs with correct functionality.

The goal is to *productionize* your Python programs and make them bulletproof while they're in use. Python has built-in modules that aid in hardening your programs. It provides facilities for debugging, optimizing, and testing to maximize the quality and performance of your programs at runtime.

Item 54: Consider Module-Scoped Code to Configure Deployment Environments

A deployment environment is a configuration in which your program runs. Every program has at least one deployment environment, the *production environment*. The goal of writing a program in the first place is to put it to work in the production environment and achieve some kind of outcome.

Writing or modifying a program requires being able to run it on the computer you use for developing. The configuration of your *development environment* may be much different from your production environment. For example, you may be writing a program for supercomputers using a Linux workstation.

Tools like `pyvenv` (see [Item 53: “Use Virtual Environments for Isolated and Reproducible Dependencies”](#)) make it easy to ensure that all environments have the same Python packages installed. The trouble is that production environments often require many external assumptions that are hard to reproduce in development environments.

For example, say you want to run your program in a web server container and give it access to a database. This means that every time you want to modify your program's code, you need to run a server container, the database must be set up properly, and your program needs the password for access. That's a very high cost if all you're trying to do is verify that a one-line change to your program works correctly.

The best way to work around these issues is to override parts of your program at startup time to provide different functionality depending on the deployment environment. For example, you could have two different `__main__` files, one for production and one for development.

[Click here to view code image](#)

```
# dev_main.py
TESTING = True
import db_connection
db = db_connection.Database()

# prod_main.py
TESTING = False
import db_connection
```

```
db = db_connection.Database()
```

The only difference between the two files is the value of the `TESTING` constant. Other modules in your program can then import the `__main__` module and use the value of `TESTING` to decide how they define their own attributes.

[Click here to view code image](#)

```
# db_connection.py
import __main__

class TestingDatabase(object):
    # ...

class RealDatabase(object):
    # ...

if __main__.TESTING:
    Database = TestingDatabase
else:
    Database = RealDatabase
```

The key behavior to notice here is that code running in module scope—not inside any function or method—is just normal Python code. You can use an `if` statement at the module level to decide how the module will define names. This makes it easy to tailor modules to your various deployment environments. You can avoid having to reproduce costly assumptions like database configurations when they aren’t needed. You can inject fake or mock implementations that ease interactive development and testing (see [Item 56: “Test Everything with `unittest`”](#)).

Note

Once your deployment environments get complicated, you should consider moving them out of Python constants (like `TESTING`) and into dedicated configuration files. Tools like the `configparser` built-in module let you maintain production configurations separate from code, a distinction that’s crucial for collaborating with an operations team.

This approach can be used for more than working around external assumptions. For example, if you know that your program must work differently based on its host platform, you can inspect the `SYS` module before defining top-level constructs in a module.

[Click here to view code image](#)

```
# db_connection.py
import sys

class Win32Database(object):
    # ...

class PosixDatabase(object):
    # ...

if sys.platform.startswith('win32'):
    Database = Win32Database
else:
```

```
Database = PosixDatabase
```

Similarly, you can use environment variables from `os.environ` to guide your module definitions.

Things to Remember

- ◆ Programs often need to run in multiple deployment environments that each have unique assumptions and configurations.
- ◆ You can tailor a module's contents to different deployment environments by using normal Python statements in module scope.
- ◆ Module contents can be the product of any external condition, including host introspection through the `sys` and `os` modules.

Item 55: Use `repr` Strings for Debugging Output

When debugging a Python program, the `print` function (or output via the `logging` built-in module) will get you surprisingly far. Python internals are often easy to access via plain attributes (see [Item 27: “Prefer Public Attributes Over Private Ones”](#)). All you need to do is `print` how the state of your program changes while it runs and see where it goes wrong.

The `print` function outputs a human-readable string version of whatever you supply it. For example, printing a basic string will print the contents of the string without the surrounding quote characters.

```
print('foo bar')
>>>
foo bar
```

This is equivalent to using the `'%s'` format string and the `%` operator.

```
print('%s' % 'foo bar')
>>>
foo bar
```

The problem is that the human-readable string for a value doesn't make it clear what the actual type of the value is. For example, notice how in the default output of `print` you can't distinguish between the types of the number `5` and the string `'5'`.

```
print(5)
print('5')

>>>
5
5
```

If you're debugging a program with `print`, these type differences matter. What you almost always want while debugging is to see the `repr` version of an object. The `repr` built-in function returns the *printable representation* of an object, which should be its most clearly understandable string representation. For built-in types, the string returned by `repr` is a valid Python expression.

```
a = '\x07'
print(repr(a))

>>>
'\x07'
```

Passing the value from `repr` to the `eval` built-in function should result in the same Python object you started with (of course, in practice, you should only use `eval` with extreme caution).

```
b = eval(repr(a))
assert a == b
```

When you're debugging with `print`, you should `repr` the value before printing to ensure that any difference in types is clear.

```
print(repr(5))
print(repr('5'))

>>>
5
'5'
```

This is equivalent to using the `'%r'` format string and the `%` operator.

```
print('%r' % 5)
print('%r' % '5')

>>>
5
'5'
```

For dynamic Python objects, the default human-readable string value is the same as the `repr` value. This means that passing a dynamic object to `print` will do the right thing, and you don't need to explicitly call `repr` on it. Unfortunately, the default value of `repr` for `object` instances isn't especially helpful. For example, here I define a simple class and then print its value:

[Click here to view code image](#)

```
class OpaqueClass(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

obj = OpaqueClass(1, 2)
print(obj)

>>>
<__main__.OpaqueClass object at 0x107880ba8>
```

This output can't be passed to the `eval` function, and it says nothing about the instance fields of the object.

There are two solutions to this problem. If you have control of the class, you can define your own `__repr__` special method that returns a string containing the Python expression that recreates the object. Here, I define that function for the class above:

[Click here to view code image](#)

```
class BetterClass(object):
    def __init__(self, x, y):
        # ...

    def __repr__(self):
        return 'BetterClass(%d, %d)' % (self.x, self.y)
```

Now, the `repr` value is much more useful.

```
obj = BetterClass(1, 2)
print(obj)

>>>
BetterClass(1, 2)
```

When you don't have control over the class definition, you can reach into the object's instance dictionary, which is stored in the `__dict__` attribute. Here, I print out the contents of an `OpaqueClass` instance:

```
obj = OpaqueClass(4, 5)
print(obj.__dict__)

>>>
{'y': 5, 'x': 4}
```

Things to Remember

- ◆ Calling `print` on built-in Python types will produce the human-readable string version of a value, which hides type information.
- ◆ Calling `repr` on built-in Python types will produce the printable string version of a value. These `repr` strings could be passed to the `eval` built-in function to get back the original value.
- ◆ `%s` in format strings will produce human-readable strings like `str.%r` will produce printable strings like `repr`.
- ◆ You can define the `__repr__` method to customize the printable representation of a class and provide more detailed debugging information.
- ◆ You can reach into any object's `__dict__` attribute to view its internals.

Item 56: Test Everything with `unittest`

Python doesn't have static type checking. There's nothing in the compiler that will ensure that your program will work when you run it. With Python you don't know whether the functions your program calls will be defined at runtime, even when their existence is evident in the source code. This dynamic behavior is a blessing and a curse.

The large numbers of Python programmers out there say it's worth it because of the productivity gained from the resulting brevity and simplicity. But most people have heard at least one horror story about Python in which a program encountered a boneheaded error at runtime.

One of the worst examples I've heard is when a `SyntaxError` was raised in production

as a side effect of a dynamic import (see [Item 52: “Know How to Break Circular Dependencies”](#)). The programmer I know who was hit by this surprising occurrence has since ruled out using Python ever again.

But I have to wonder, why wasn't the code tested before the program was deployed to production? Type safety isn't everything. You should always test your code, regardless of what language it's written in. However, I'll admit that the big difference between Python and many other languages is that the only way to have *any* confidence in a Python program is by writing tests. There is no veil of static type checking to make you feel safe.

Luckily, the same dynamic features that prevent static type checking in Python also make it extremely easy to write tests for your code. You can use Python's dynamic nature and easily overridable behaviors to implement tests and ensure that your programs work as expected.

You should think of tests as an insurance policy on your code. Good tests give you confidence that your code is correct. If you refactor or expand your code, tests make it easy to identify how behaviors have changed. It sounds counter-intuitive, but having good tests actually makes it easier to modify Python code, not harder.

The simplest way to write tests is to use the `unittest` built-in module. For example, say you have the following utility function defined in `utils.py`:

[Click here to view code image](#)

```
# utils.py
def to_str(data):
    if isinstance(data, str):
        return data
    elif isinstance(data, bytes):
        return data.decode('utf-8')
    else:
        raise TypeError('Must supply str or bytes, '
                        'found: %r' % data)
```

To define tests, I create a second file named `test_utils.py` or `utils_test.py` that contains tests for each behavior I expect.

[Click here to view code image](#)

```
# utils_test.py
from unittest import TestCase, main
from utils import to_str

class UtilsTestCase(TestCase):
    def test_to_str_bytes(self):
        self.assertEqual('hello', to_str(b'hello'))

    def test_to_str_str(self):
        self.assertEqual('hello', to_str('hello'))

    def test_to_str_bad(self):
        self.assertRaises(TypeError, to_str, object())

if __name__ == '__main__':
    main()
```

Tests are organized into `TestCase` classes. Each test is a method beginning with the

word `test`. If a test method runs without raising any kind of `Exception` (including `AssertionError` from `assert` statements), then the test is considered to have passed successfully.

The `TestCase` class provides helper methods for making assertions in your tests, such as `assertEqual` for verifying equality, `assertTrue` for verifying Boolean expressions, and `assertRaises` for verifying that exceptions are raised when appropriate (see `help(TestCase)` for more). You can define your own helper methods in `TestCase` subclasses to make your tests more readable; just ensure that your method names don't begin with the word `test`.

Note

Another common practice when writing tests is to use mock functions and classes to stub out certain behaviors. For this purpose, Python 3 provides the `unittest.mock` built-in module, which is also available for Python 2 as an open source package.

Sometimes, your `TestCase` classes need to set up the test environment before running test methods. To do this, you can override the `setUp` and `tearDown` methods. These methods are called before and after each test method, respectively, and they let you ensure that each test runs in isolation (an important best practice of proper testing). For example, here I define a `TestCase` that creates a temporary directory before each test and deletes its contents after each test finishes:

[Click here to view code image](#)

```
class MyTest(TestCase):
    def setUp(self):
        self.test_dir = TemporaryDirectory()
    def tearDown(self):
        self.test_dir.cleanup()
    # Test methods follow
    # ...
```

I usually define one `TestCase` for each set of related tests. Sometimes I have one `TestCase` for each function that has many edge cases. Other times, a `TestCase` spans all functions in a single module. I'll also create one `TestCase` for testing a single class and all of its methods.

When programs get complicated, you'll want additional tests for verifying the interactions between your modules, instead of only testing code in isolation. This is the difference between *unit tests* and *integration tests*. In Python, it's important to write both types of tests for exactly the same reason: You have no guarantee that your modules will actually work together unless you prove it.

Note

Depending on your project, it can also be useful to define data-driven tests or organize tests into different suites of related functionality. For these purposes, code coverage reports, and other advanced use cases, the `nose` (<http://nose.readthedocs.org/>) and `pytest` (<http://pytest.org/>) open source packages can be especially helpful.

Things to Remember

- ◆ The only way to have confidence in a Python program is to write tests.
- ◆ The `unittest` built-in module provides most of the facilities you'll need to write good tests.
- ◆ You can define tests by subclassing `TestCase` and defining one method per behavior you'd like to test. Test methods on `TestCase` classes must start with the word `test`.
- ◆ It's important to write both unit tests (for isolated functionality) and integration tests (for modules that interact).

Item 57: Consider Interactive Debugging with `pdb`

Everyone encounters bugs in their code while developing programs. Using the `print` function can help you track down the source of many issues (see [Item 55: “Use repr Strings for Debugging Output”](#)). Writing tests for specific cases that cause trouble is another great way to isolate problems (see [Item 56: “Test Everything with `unittest`”](#)).

But these tools aren't enough to find every root cause. When you need something more powerful, it's time to try Python's built-in *interactive debugger*. The debugger lets you inspect program state, print local variables, and step through a Python program one statement at a time.

In most other programming languages, you use a debugger by specifying what line of a source file you'd like to stop on, then execute the program. In contrast, with Python the easiest way to use the debugger is by modifying your program to directly initiate the debugger just before you think you'll have an issue worth investigating. There is no difference between running a Python program under a debugger and running it normally.

To initiate the debugger, all you have to do is import the `pdb` built-in module and run its `set_trace` function. You'll often see this done in a single line so programmers can comment it out with a single `#` character.

[Click here to view code image](#)

```
def complex_func(a, b, c):  
    # ...  
    import pdb; pdb.set_trace()
```

As soon as this statement runs, the program will pause its execution. The terminal that

started your program will turn into an interactive Python shell.

[Click here to view code image](#)

```
-> import pdb; pdb.set_trace()  
(Pdb)
```

At the `(Pdb)` prompt, you can type in the name of local variables to see their values printed out. You can see a list of all local variables by calling the `locals` built-in function. You can import modules, inspect global state, construct new objects, run the `help` built-in function, and even modify parts of the program—whatever you need to do to aid in your debugging. In addition, the debugger has three commands that make inspecting the running program easier.

- **bt**: Print the traceback of the current execution call stack. This lets you figure out where you are in your program and how you arrived at the `pdb.set_trace` trigger point.
- **up**: Move your scope up the function call stack to the caller of the current function. This allows you to inspect the local variables in higher levels of the call stack.
- **down**: Move your scope back down the function call stack one level.

Once you're done inspecting the current state, you can use debugger commands to resume the program's execution under precise control.

- **step**: Run the program until the next line of execution in the program, then return control back to the debugger. If the next line of execution includes calling a function, the debugger will stop in the function that was called.
- **next**: Run the program until the next line of execution in the current function, then return control back to the debugger. If the next line of execution includes calling a function, the debugger will not stop until the called function has returned.
- **return**: Run the program until the current function returns, then return control back to the debugger.
- **continue**: Continue running the program until the next breakpoint (or `set_trace` is called again).

Things to Remember

- ◆ You can initiate the Python interactive debugger at a point of interest directly in your program with the `import pdb; pdb.set_trace()` statements.
- ◆ The Python debugger prompt is a full Python shell that lets you inspect and modify the state of a running program.
- ◆ `pdb` shell commands let you precisely control program execution, allowing you to alternate between inspecting program state and progressing program execution.

Item 58: Profile Before Optimizing

The dynamic nature of Python causes surprising behaviors in its runtime performance. Operations you might assume are slow are actually very fast (string manipulation, generators). Language features you might assume are fast are actually very slow (attribute access, function calls). The true source of slowdowns in a Python program can be obscure.

The best approach is to ignore your intuition and directly measure the performance of a program before you try to optimize it. Python provides a built-in *profiler* for determining which parts of a program are responsible for its execution time. This lets you focus your optimization efforts on the biggest sources of trouble and ignore parts of the program that don't impact speed.

For example, say you want to determine why an algorithm in your program is slow. Here, I define a function that sorts a list of data using an insertion sort:

[Click here to view code image](#)

```
def insertion_sort(data):
    result = []
    for value in data:
        insert_value(result, value)
    return result
```

The core mechanism of the insertion sort is the function that finds the insertion point for each piece of data. Here, I define an extremely inefficient version of the `insert_value` function that does a linear scan over the input array:

[Click here to view code image](#)

```
def insert_value(array, value):
    for i, existing in enumerate(array):
        if existing > value:
            array.insert(i, value)
    return
array.append(value)
```

To profile `insertion_sort` and `insert_value`, I create a data set of random numbers and define a `test` function to pass to the profiler.

[Click here to view code image](#)

```
from random import randint

max_size = 10**4
data = [randint(0, max_size) for _ in range(max_size)]
test = lambda: insertion_sort(data)
```

Python provides two built-in profilers, one that is pure Python (`profile`) and another that is a C-extension module (`cProfile`). The `cProfile` built-in module is better because of its minimal impact on the performance of your program while it's being profiled. The pure-Python alternative imposes a high overhead that will skew the results.

Note

When profiling a Python program, be sure that what you're measuring is the code itself and not any external systems. Beware of functions that access the network or resources on disk. These may appear to have a large impact on your program's execution time because of the slowness of the underlying systems. If your program uses a cache to mask the latency of slow resources like these, you should also ensure that it's properly warmed up before you start profiling.

Here, I instantiate a `Profile` object from the `cProfile` module and run the test function through it using the `runcall` method:

```
profiler = Profile()
profiler.runcall(test)
```

Once the test function has finished running, I can extract statistics about its performance using the `pstats` built-in module and its `Stats` class. Various methods on a `Stats` object adjust how to select and sort the profiling information to show only the things you care about.

```
stats = Stats(profiler)
stats.strip_dirs()
stats.sort_stats('cumulative')
stats.print_stats()
```

The output is a table of information organized by function. The data sample is taken only from the time the profiler was active, during the `runcall` method above.

[Click here to view code image](#)

```
>>>
20003 function calls in 1.812 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000   0.000   1.812    1.812 main.py:34(<lambda>)
      1   0.003   0.003   1.812    1.812 main.py:10(insertion_sort)
    10000  1.797   0.000   1.810    0.000 main.py:20(insert_value)
     9992   0.013   0.000   0.013    0.000 {method 'insert' of 'list'
objects}
      8   0.000   0.000   0.000    0.000 {method 'append' of 'list'
objects}
      1   0.000   0.000   0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}
```

Here's a quick guide to what the profiler statistics columns mean:

- **ncalls**: The number of calls to the function during the profiling period.
- **tottime**: The number of seconds spent executing the function, excluding time spent executing other functions it calls.
- **tottime percall**: The average number of seconds spent in the function each time it was called, excluding time spent executing other functions it calls. This is **tottime** divided by **ncalls**.

- **cumtime**: The cumulative number of seconds spent executing the function, including time spent in all other functions it calls.
- **cumtime percall**: The average number of seconds spent in the function each time it was called, including time spent in all other functions it calls. This is **cumtime** divided by **ncalls**.

Looking at the profiler statistics table above, I can see that the biggest use of CPU in my test is the cumulative time spent in the `insert_value` function. Here, I redefine that function to use the `bisect` built-in module (see [Item 46: “Use Built-in Algorithms and Data Structures”](#)):

[Click here to view code image](#)

```
from bisect import bisect_left

def insert_value(array, value):
    i = bisect_left(array, value)
    array.insert(i, value)
```

I can run the profiler again and generate a new table of profiler statistics. The new function is much faster, with a cumulative time spent that is nearly 100× smaller than the previous `insert_value` function.

[Click here to view code image](#)

```
>>>
30003 function calls in 0.028 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1     0.000      0.000      0.028      0.028 main.py:34(<lambda>)
      1     0.002      0.002      0.028      0.028 main.py:10(insertion_sort)
    10000     0.005      0.000      0.026      0.000 main.py:112(insert_value)
    10000     0.014      0.000      0.014      0.000 {method 'insert' of 'list'
objects}
    10000     0.007      0.000      0.007      0.000 {built-in method bisect_left}
      1     0.000      0.000      0.000      0.000 {method 'disable' of
'_lsprof.Profiler' objects}
```

Sometimes, when you’re profiling an entire program, you’ll find that a common utility function is responsible for the majority of execution time. The default output from the profiler makes this situation difficult to understand because it doesn’t show how the utility function is called by many different parts of your program.

For example, here the `my_utility` function is called repeatedly by two different functions in the program:

```
def my_utility(a, b):
    # ...

def first_func():
    for _ in range(1000):
        my_utility(4, 5)

def second_func():
    for _ in range(10):
```

```

my_utility(1, 3)

def my_program():
    for _ in range(20):
        first_func()
        second_func()

```

Profiling this code and using the default `print_stats` output will generate output statistics that are confusing.

[Click here to view code image](#)

```

>>>
20242 function calls in 0.208 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1      0.000      0.000      0.208      0.208 main.py:176(my_program)
      20      0.005      0.000      0.206      0.010 main.py:168(first_func)
    20200      0.203      0.000      0.203      0.000 main.py:161(my_utility)
      20      0.000      0.000      0.002      0.000 main.py:172(second_func)
      1      0.000      0.000      0.000      0.000 {method 'disable' of
'_lsprof.Profiler' objects}

```

The `my_utility` function is clearly the source of most execution time, but it's not immediately obvious why that function is called so much. If you search through the program's code, you'll find multiple call sites for `my_utility` and still be confused.

To deal with this, the Python profiler provides a way of seeing which callers contributed to the profiling information of each function.

```
stats.print_callers()
```

This profiler statistics table shows functions called on the left and who was responsible for making the call on the right. Here, it's clear that `my_utility` is most used by `first_func`:

[Click here to view code image](#)

```

>>>
Ordered by: cumulative time

Function                                     was called by...
                                     ncalls      tottime  cumtime
main.py:176(my_program)                   <-
main.py:168(first_func)                   <-      20      0.005    0.206  main.py:
main.py:161(my_utility)                   <-    20000    0.202    0.202  main.py:
                                     200      0.002    0.002  main.py:
main.py:172(second_func)                   <-      20      0.000    0.002  main.py:

```

Things to Remember

- ◆ It's important to profile Python programs before optimizing because the source of slowdowns is often obscure.
- ◆ Use the `cProfile` module instead of the `profile` module because it provides more accurate profiling information.

- ◆ The `Profile` object's `runcall` method provides everything you need to profile a tree of function calls in isolation.
- ◆ The `Stats` object lets you select and print the subset of profiling information you need to see to understand your program's performance.

Item 59: Use `tracemalloc` to Understand Memory Usage and Leaks

Memory management in the default implementation of Python, CPython, uses reference counting. This ensures that as soon as all references to an object have expired, the referenced object is also cleared. CPython also has a built-in cycle detector to ensure that self-referencing objects are eventually garbage collected.

In theory, this means that most Python programmers don't have to worry about allocating or deallocating memory in their programs. It's taken care of automatically by the language and the CPython runtime. However, in practice, programs eventually do run out of memory due to held references. Figuring out where your Python programs are using or leaking memory proves to be a challenge.

The first way to debug memory usage is to ask the `gc` built-in module to list every object currently known by the garbage collector. Although it's quite a blunt tool, this approach does let you quickly get a sense of where your program's memory is being used.

Here, I run a program that wastes memory by keeping references. It prints out how many objects were created during execution and a small sample of allocated objects.

[Click here to view code image](#)

```
# using_gc.py
import gc
found_objects = gc.get_objects()
print('%d objects before' % len(found_objects))

import waste_memory
x = waste_memory.run()
found_objects = gc.get_objects()
print('%d objects after' % len(found_objects))
for obj in found_objects[:3]:
    print(repr(obj)[:100])

>>>
4756 objects before
14873 objects after
<waste_memory.MyObject object at 0x1063f6940>
<waste_memory.MyObject object at 0x1063f6978>
<waste_memory.MyObject object at 0x1063f69b0>
```

The problem with `gc.get_objects` is that it doesn't tell you anything about *how* the objects were allocated. In complicated programs, a specific class of object could be allocated many different ways. The overall number of objects isn't nearly as important as identifying the code responsible for allocating the objects that are leaking memory.

Python 3.4 introduces a new `tracemalloc` built-in module for solving this problem.

`tracemalloc` makes it possible to connect an object back to where it was allocated. Here, I print out the top three memory usage offenders in a program using `tracemalloc`:

[Click here to view code image](#)

```
# top_n.py
import tracemalloc
tracemalloc.start(10) # Save up to 10 stack frames

time1 = tracemalloc.take_snapshot()
import waste_memory
x = waste_memory.run()
time2 = tracemalloc.take_snapshot()

stats = time2.compare_to(time1, 'lineno')
for stat in stats[:3]:
    print(stat)

>>>
waste_memory.py:6: size=2235 KiB (+2235 KiB), count=29981 (+29981),
average=76 B
waste_memory.py:7: size=869 KiB (+869 KiB), count=10000 (+10000), average=89
B
waste_memory.py:12: size=547 KiB (+547 KiB), count=10000 (+10000), average=56
B
```

It's immediately clear which objects are dominating my program's memory usage and where in the source code they were allocated.

The `tracemalloc` module can also print out the full stack trace of each allocation (up to the number of frames passed to the `start` method). Here, I print out the stack trace of the biggest source of memory usage in the program:

[Click here to view code image](#)

```
# with_trace.py
# ...
stats = time2.compare_to(time1, 'traceback')
top = stats[0]
print('\n'.join(top.traceback.format()))

>>>
File "waste_memory.py", line 6
    self.x = os.urandom(100)
File "waste_memory.py", line 12
    obj = MyObject()
File "waste_memory.py", line 19
    deep_values.append(get_data())
File "with_trace.py", line 10
    x = waste_memory.run()
```

A stack trace like this is most valuable for figuring out which particular usage of a common function is responsible for memory consumption in a program.

Unfortunately, Python 2 doesn't provide the `tracemalloc` built-in module. There are open source packages for tracking memory usage in Python 2 (such as `heapy`), though they do not fully replicate the functionality of `tracemalloc`.

Things to Remember

- ◆ It can be difficult to understand how Python programs use and leak memory.
- ◆ The `gc` module can help you understand which objects exist, but it has no information about how they were allocated.
- ◆ The `tracemalloc` built-in module provides powerful tools for understanding the source of memory usage.
- ◆ `tracemalloc` is only available in Python 3.4 and above.